RAG Failures and Fixes

1. Introduction

A basic RAG pipeline works well in a demo. With real documents and real users, answers start to go wrong. Most problems come from **five failures**, and each one has known fixes.

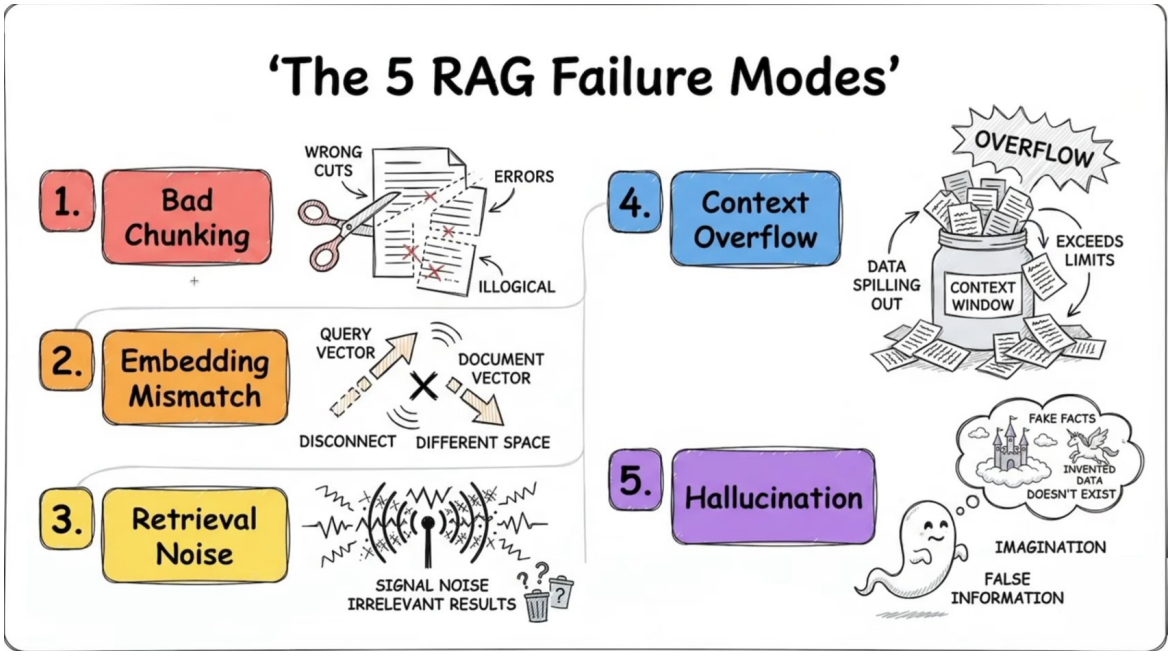


Figure 1: The five RAG failure modes.

2. Failure 1: Bad Chunking

A cut in the wrong place splits one idea into two chunks. Each half alone is wrong or useless, and so is the answer built on it.

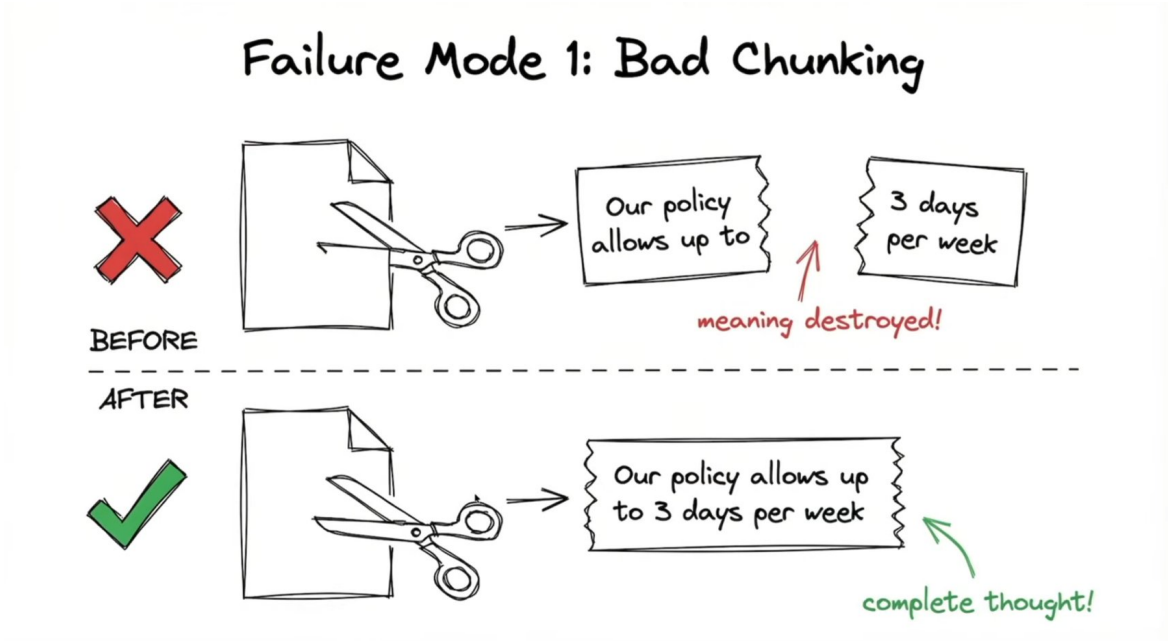


Figure 2: One sentence cut in two loses its meaning; kept whole, it keeps it.

How to fix it

- **Tune size and overlap:** test a few chunk sizes and 10-20% overlap on your real questions.
- **Pick the right splitter:** recursive splitting follows paragraphs and sentences; semantic splitting cuts where the topic changes.
- **Parent-child chunks:** search with small chunks (precise), but give the LLM the bigger parent chunk (context).

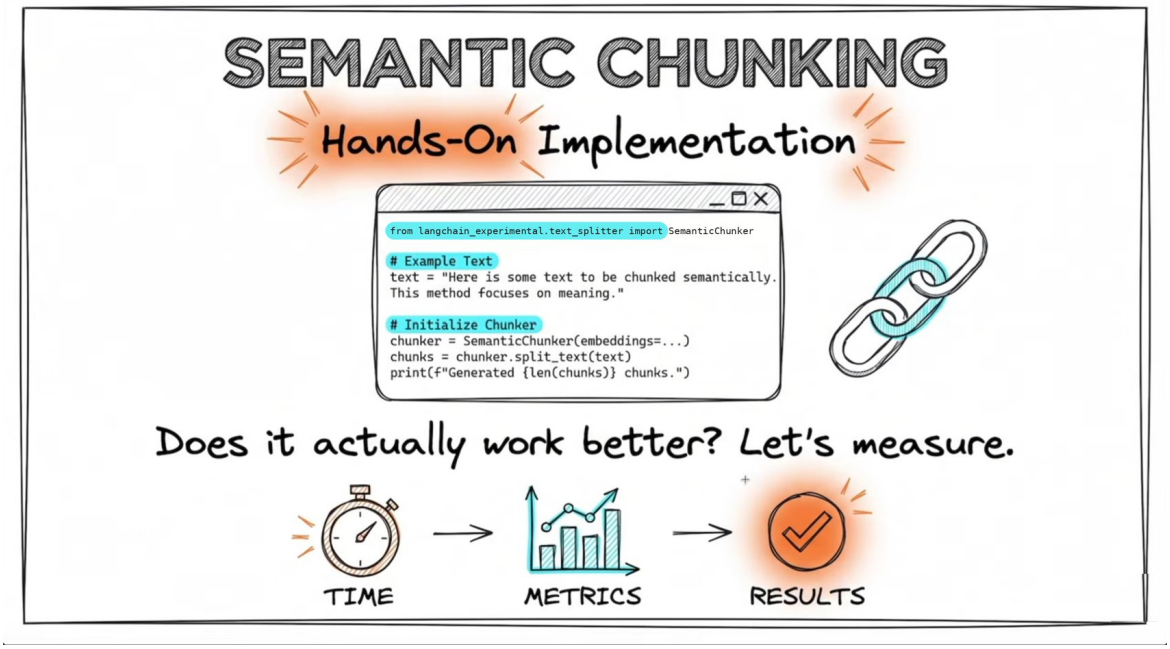


Figure 3: Semantic chunking in LangChain. Does it work better? We measure.

> **Observation:** On a structured document (clear headers and sections), recursive chunking returned cleaner, more focused chunks. Semantic chunking does better on unstructured text, where topic changes are not marked by headings.

3. Failure 2: Embedding Mismatch

An embedding captures **meaning**, not exact strings. Product codes, error codes, acronyms and names carry little meaning, so vector search often returns the wrong documents.

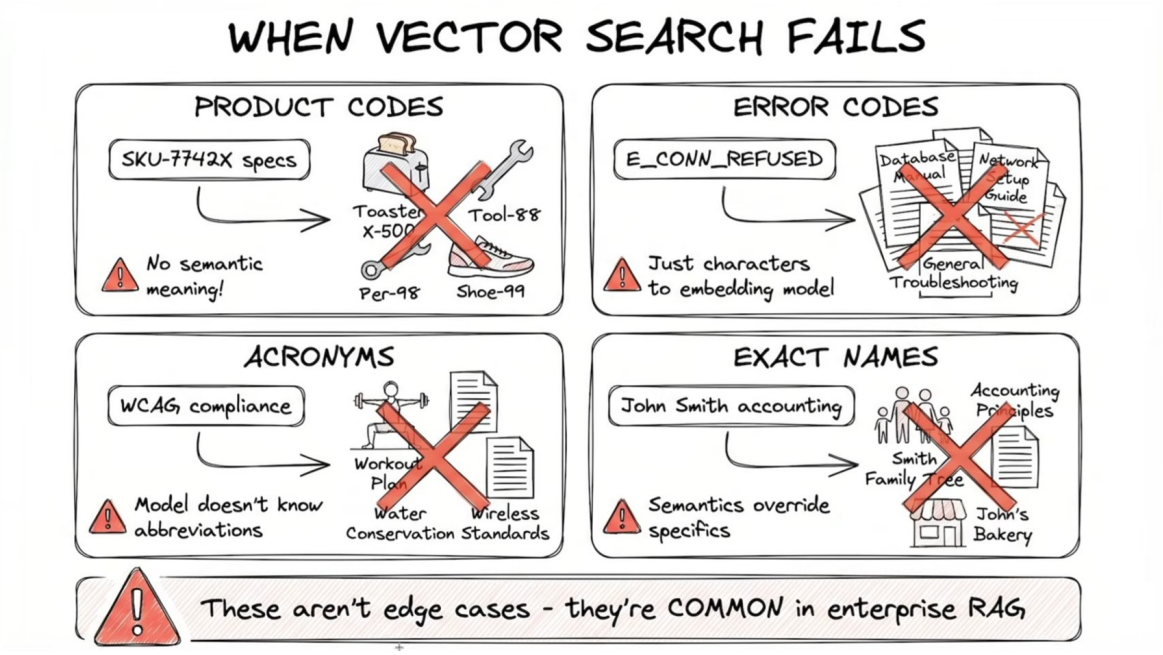


Figure 4: Four common cases where vector search fails.

The fix is **hybrid search**: run two searches and merge them. Vector search finds meaning, BM25 finds exact words.

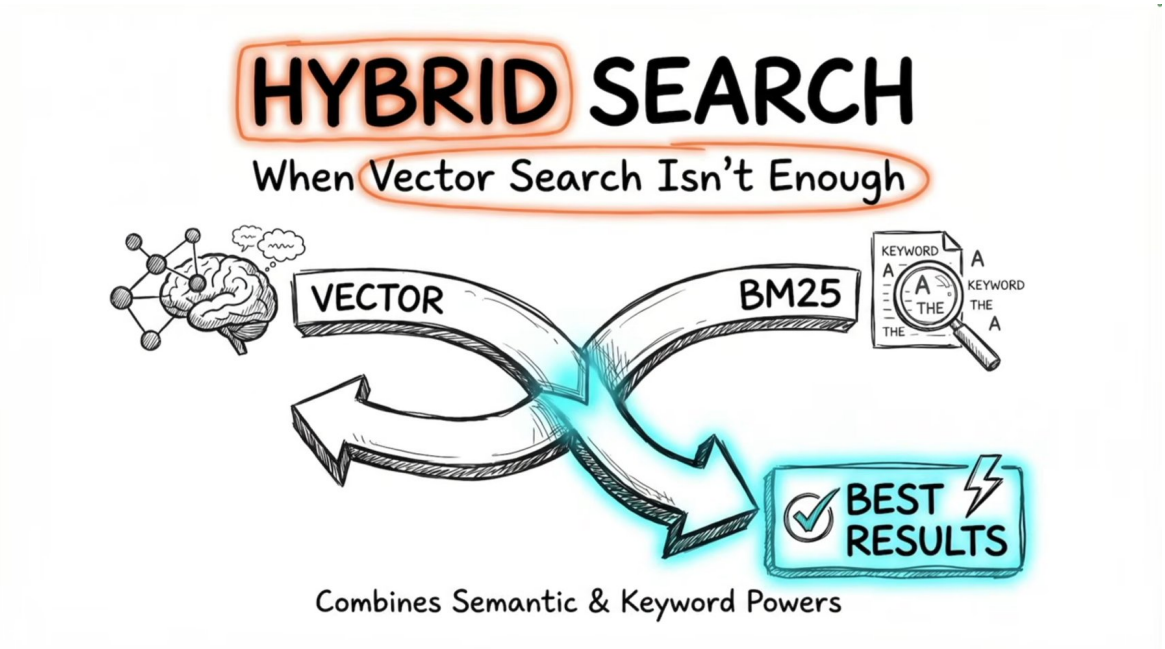


Figure 5: Hybrid search combines semantic and keyword search.

- **Vector search:** turns the question into a vector and finds the chunks with the closest vectors, so it matches meaning, not exact words. "Late check-out" can find "departure after 12:00".
- **BM25:** classic keyword search. It scores a document by how often the query words appear in it, and gives more weight to rare words.

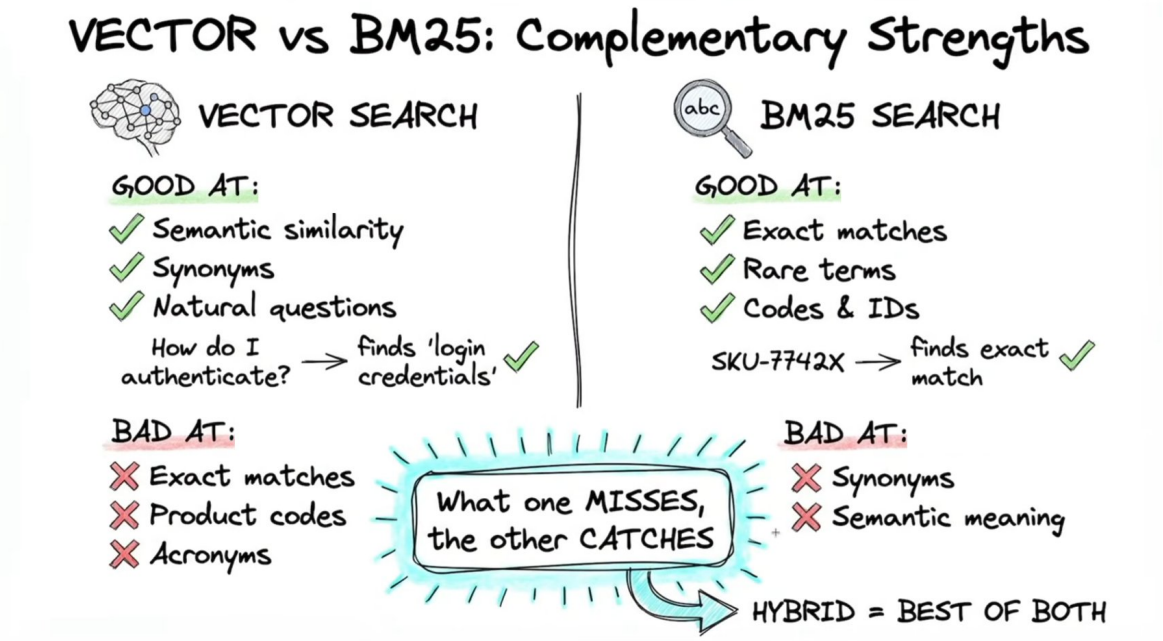


Figure 6: What one search misses, the other catches.

- **RRF (Reciprocal Rank Fusion):** each search returns a ranked list. Each document gets 1/(k + rank) from every list (k is usually 60), and the scores are added. A document that ranks well in both lists rises to the top. RRF uses ranks, not raw scores, so we never compare cosine scores with BM25 scores.

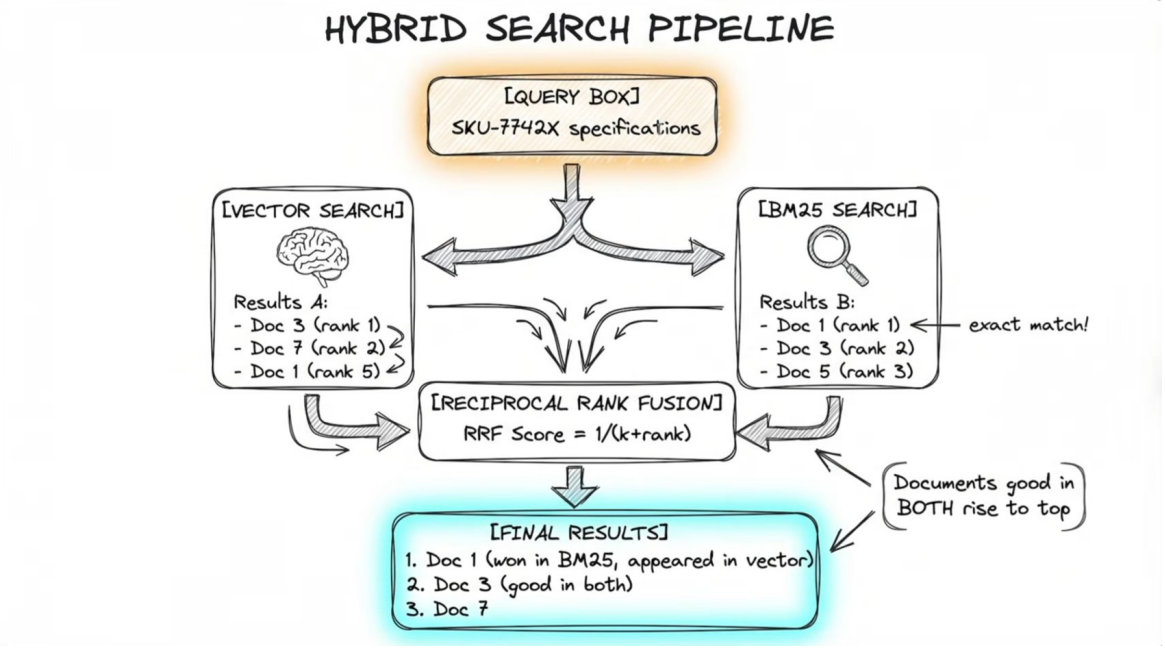


Figure 7: Both searches run, then RRF merges their ranked lists.

In LangChain this takes three components: a vector retriever, a BM25 retriever, and an ensemble retriever that merges them with weighted RRF.

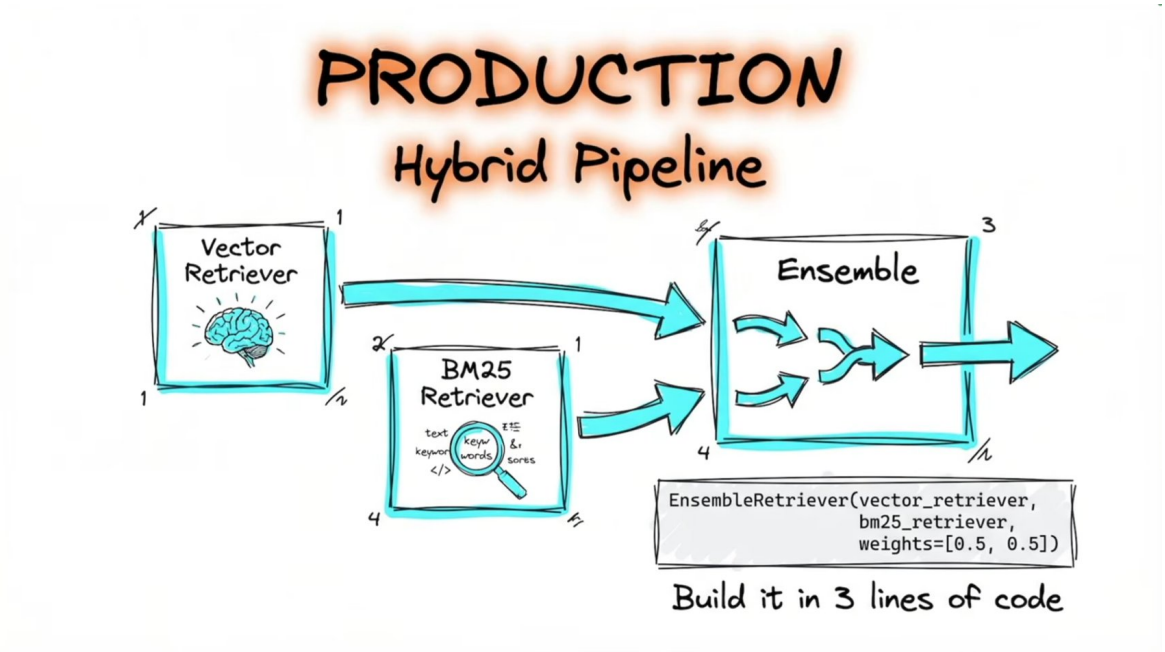


Figure 8: A production hybrid pipeline.

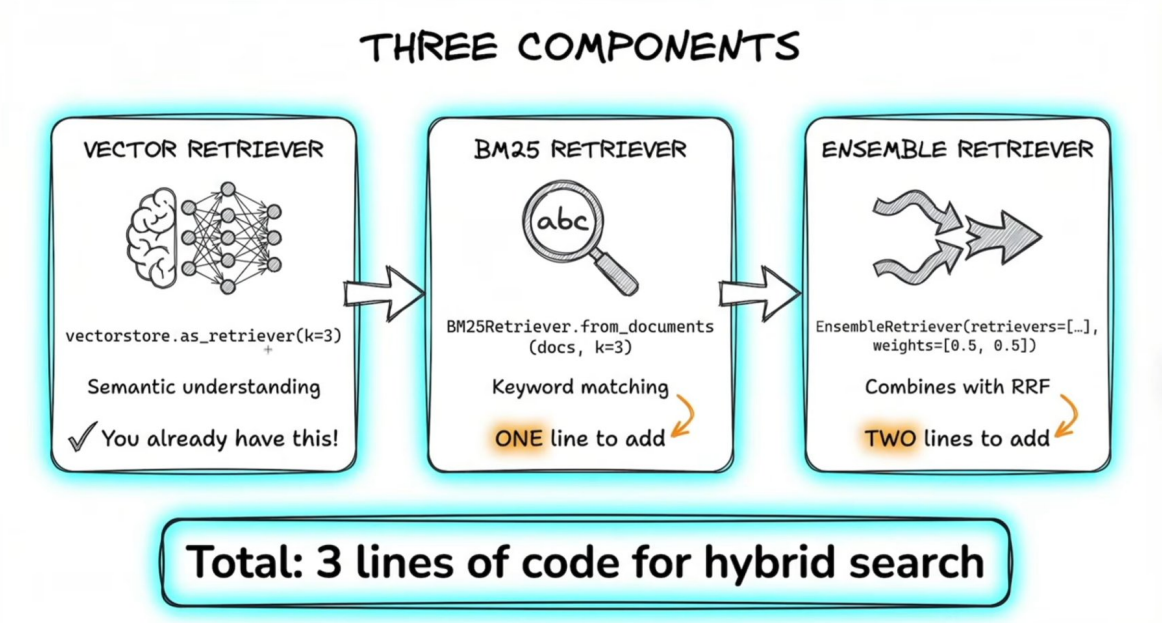


Figure 9: The three components in code.

Tuning the weights

Start at 50/50. Move toward BM25 if users search codes and IDs, toward vector for natural questions. Choose the weights with test questions, not by feel.

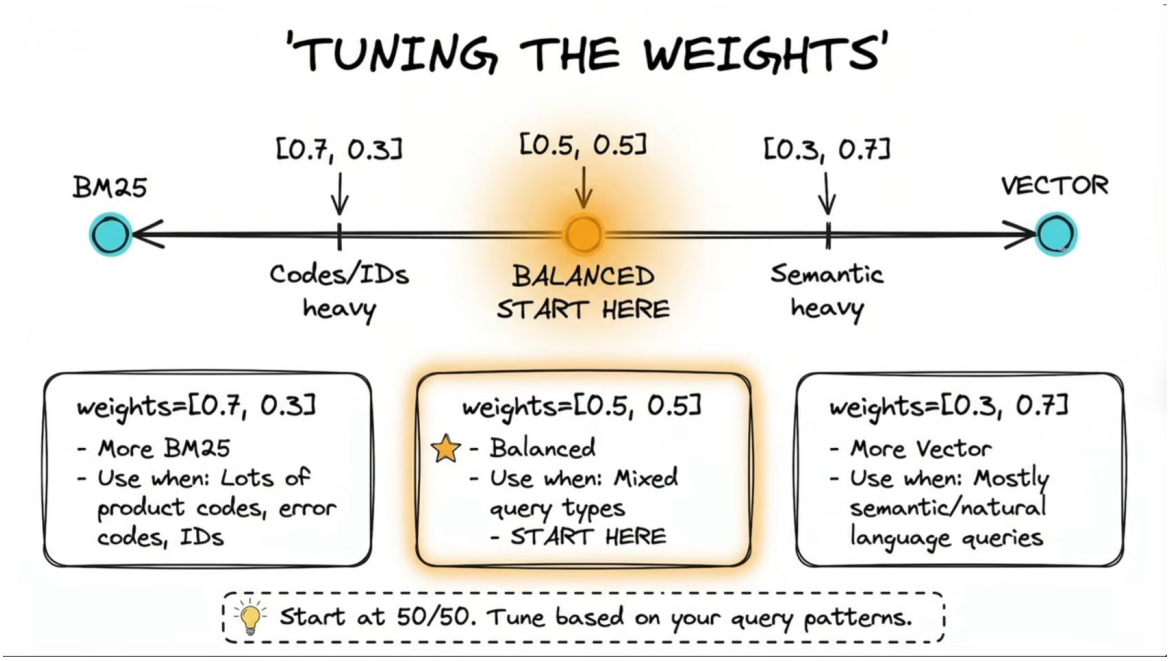


Figure 10: Weights move the balance between BM25 and vector search.

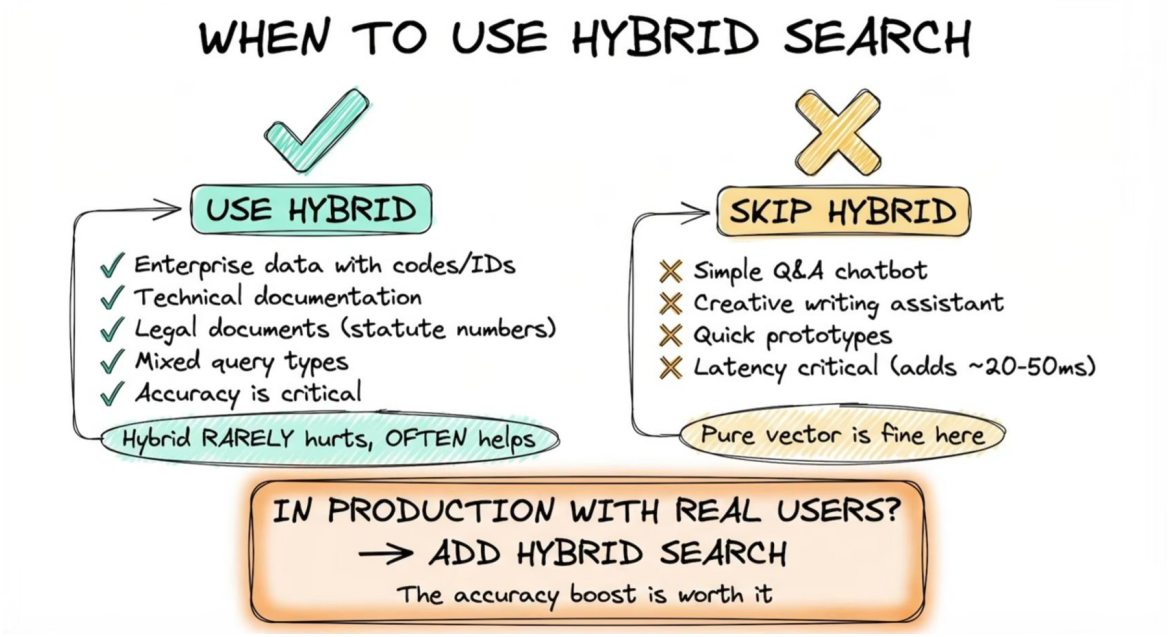


Figure 11: When hybrid search is worth it, and when pure vector is enough.

Production notes

- **Rebuild BM25:** LangChain's BM25 retriever lives in memory, so rebuild it when documents change. Search engines such as Elasticsearch or OpenSearch update it for us.
- **Clean the words:** LangChain's BM25 only splits text on spaces by default. Lowercase the text (and for Greek, remove accents) so "Πισίνα" matches "πισινα".

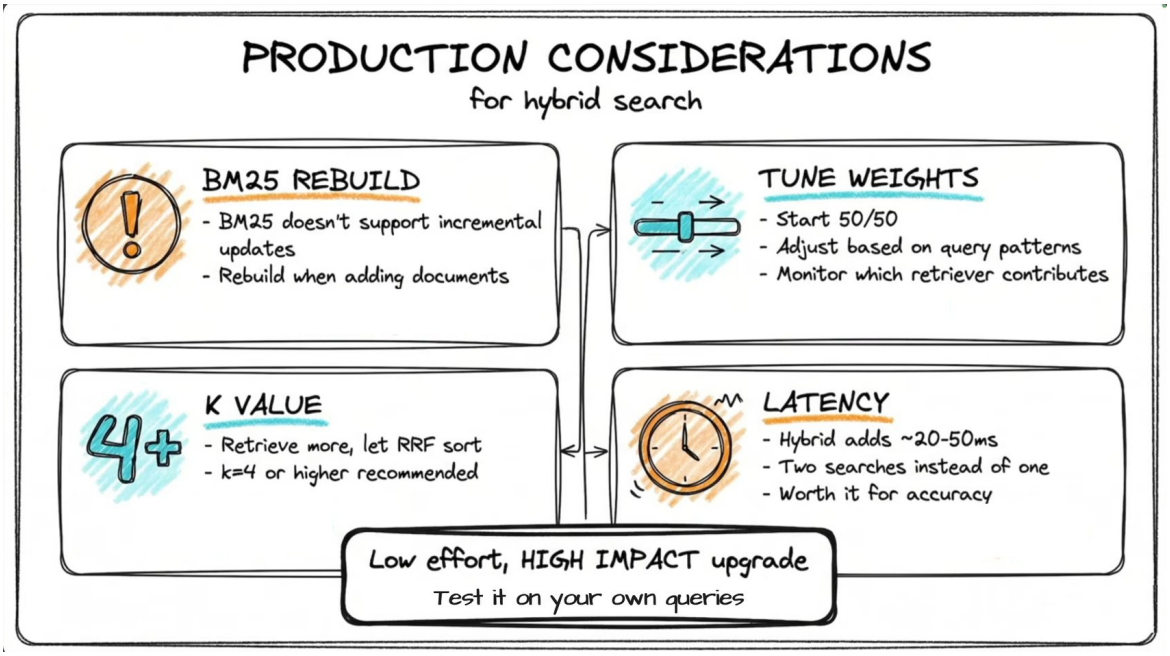


Figure 12: What to watch when hybrid search goes to production.

4. Failure 3: Retrieval Noise

The top-k results hold chunks that look similar to the question but do not answer it. They push the right chunk out of the top-k and distract the LLM.

How to fix it

- **Reranking:** retrieve many chunks (e.g. 20), then a reranker scores each one against the question and keeps the best 3-5. A reranker (cross-encoder) reads the question and the chunk together, so it judges relevance much better than vector distance. It is slower, so we run it only on a few candidates. Examples: Cohere Rerank, bge-reranker.
- **Contextual compression:** after retrieval, keep only the parts of each chunk that relate to the question.
- **Multi-query:** the LLM rewrites the question in a few ways, we search with each and merge the results. It helps with short or vague questions.
- **Filters and thresholds:** search only the right documents with metadata (e.g. hotel, language, date), and drop chunks below a minimum similarity score.

> **Observation:** Contextual compression gave clearly better answers on documents where the answer was buried inside long chunks. The LLM-based version costs one extra LLM call per chunk, so use it when precision matters. A cheaper option filters chunks with embeddings.

5. Failure 4: Context Overflow

Too much text goes into the prompt. It may pass the model's limit, it costs more, and models often miss facts placed in the middle of a long context.

How to fix it

- **Send fewer, better chunks:** rerank and keep a small top-k.
- **Compress:** contextual compression removes text the LLM does not need.
- **Limit chat history:** keep the last few turns or a short summary.

Cost control: token budgeting

Before each LLM call, estimate the tokens. If the request is over budget, block or trim it; otherwise proceed. A normal question with its context is 2-3K tokens, while one user who pastes 50K tokens costs about 20 times more. Track budgets **per user** and **per endpoint**. Use it whenever users send free text of unpredictable length.

> **Careful:** token budgeting protects our cost, it does not fix overflow. Fewer chunks and compression do.

6. Failure 5: Hallucination

The LLM writes facts that are not in the retrieved text. It happens most when the context is missing or weak, so the model fills the gap.

How to fix it

- **Strict prompt:** "Answer only from the context. If the answer is not there, say you do not know."
- **Show sources:** cite the document and page from the metadata, so users can check the answer.
- **Low temperature:** 0 to 0.2, for less creative and more faithful answers.
- **Refuse when retrieval is weak:** if the best chunk scores below a threshold, do not answer.
- **Measure faithfulness:** on a set of test questions, check that every claim in the answer appears in the context. Tools like RAGAS automate this.

> **Careful:** many hallucinations start as retrieval failures. If the right chunk is never retrieved, no prompt can fix the answer. Fix failures 1-3 first.

7. Observability

Here we use **LangSmith**, a platform from the LangChain team for tracing, monitoring and evaluating LLM apps. It records every step of each run, so we can see what happened and where it went wrong.

What it is: seeing what the system does at every step, not only the final answer. It has three parts: **traces** (what happened), **metrics** (tokens, time, cost) and **evals** (was it good?).

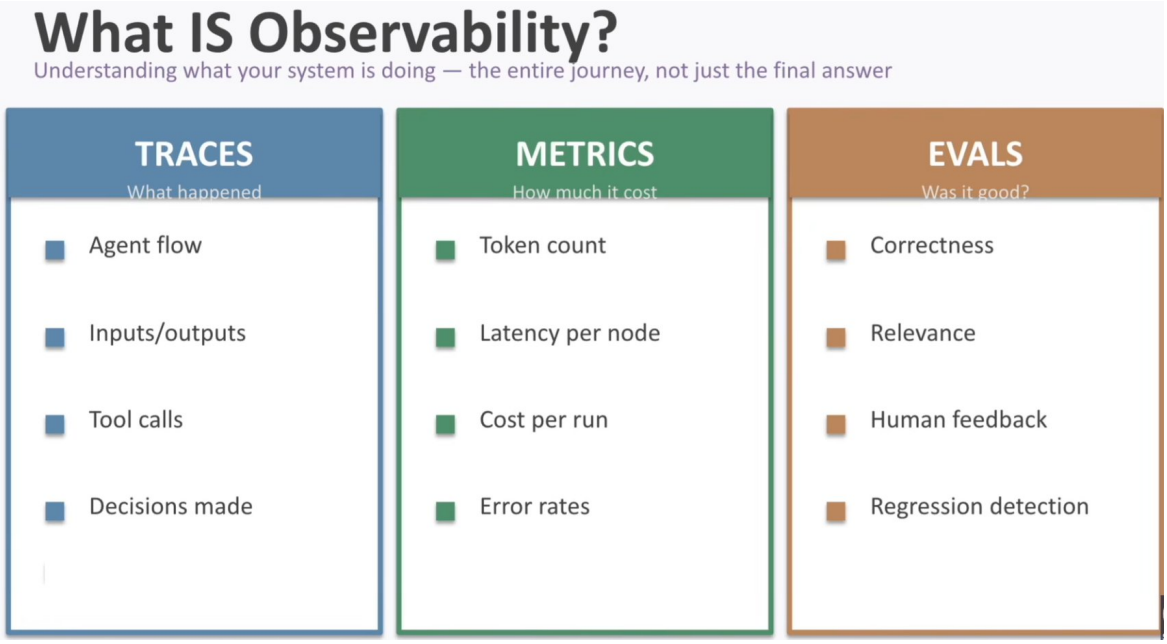


Figure 13: The three parts of observability.

Why we need it: LLM apps fail differently from normal software. There is no stack trace, the same input can give different answers, errors cascade from step to step, and a wrong answer does not crash anything.

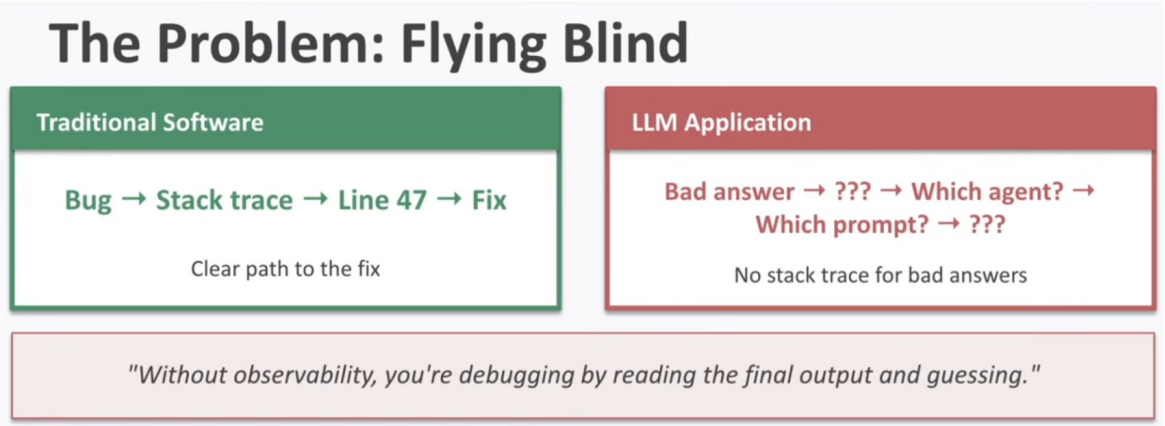


Figure 14: Without observability we only see the final answer and guess.

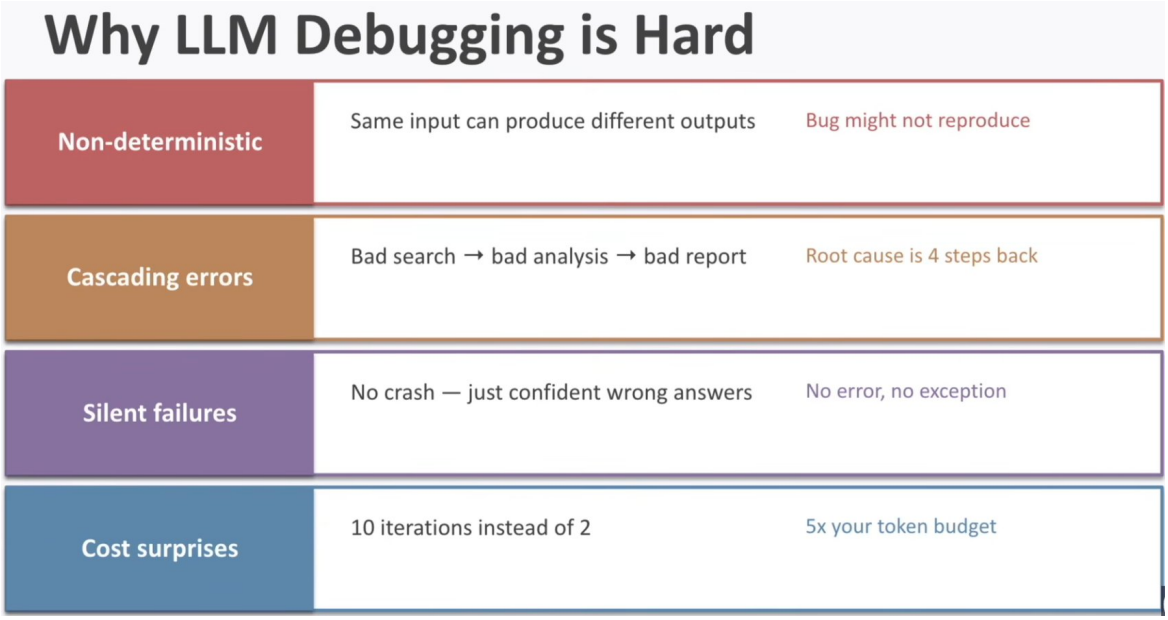


Figure 15: Why debugging LLM apps is hard.

For RAG, a trace shows the question, the retrieved chunks, the final prompt, the answer, the tokens and the time. It tells us where the failure is: **wrong chunks** means a retrieval failure (failures 1-3); **right chunks but a wrong answer** means a generation failure (failure 5).

How: LangSmith traces every LangChain call once we set two environment variables (*LANGSMITH_TRACING=true* and *LANGSMITH_API_KEY*), with no code changes. Langfuse is an open-source alternative we can host ourselves.

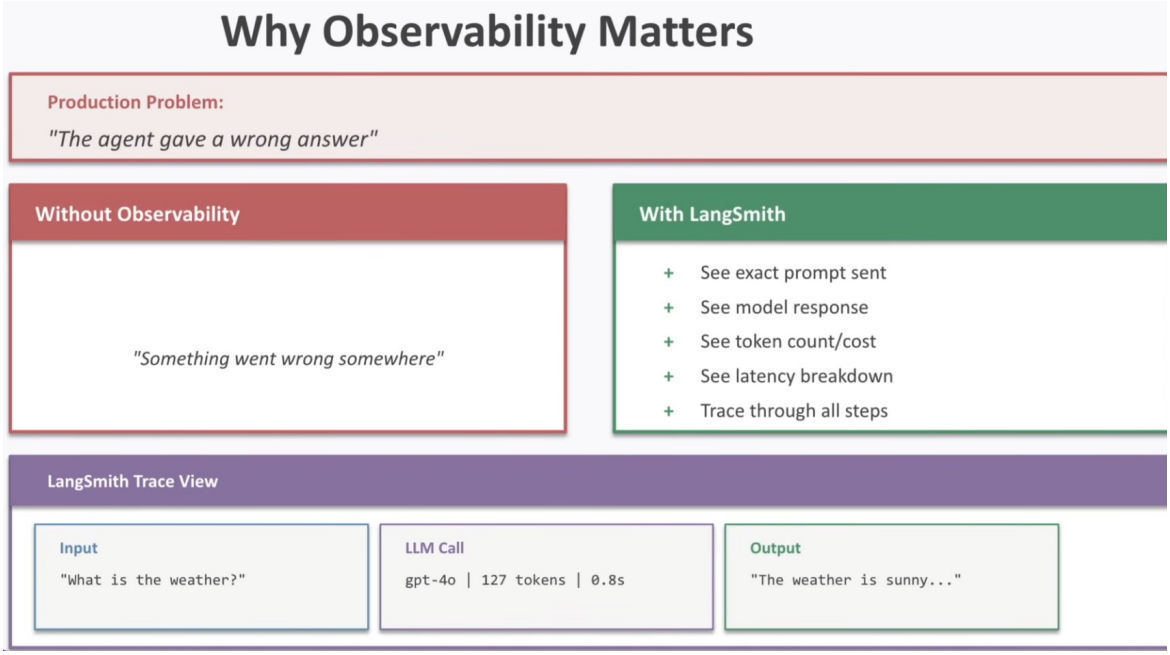


Figure 16: What a LangSmith trace shows.

Add observability from the start, not after something breaks.

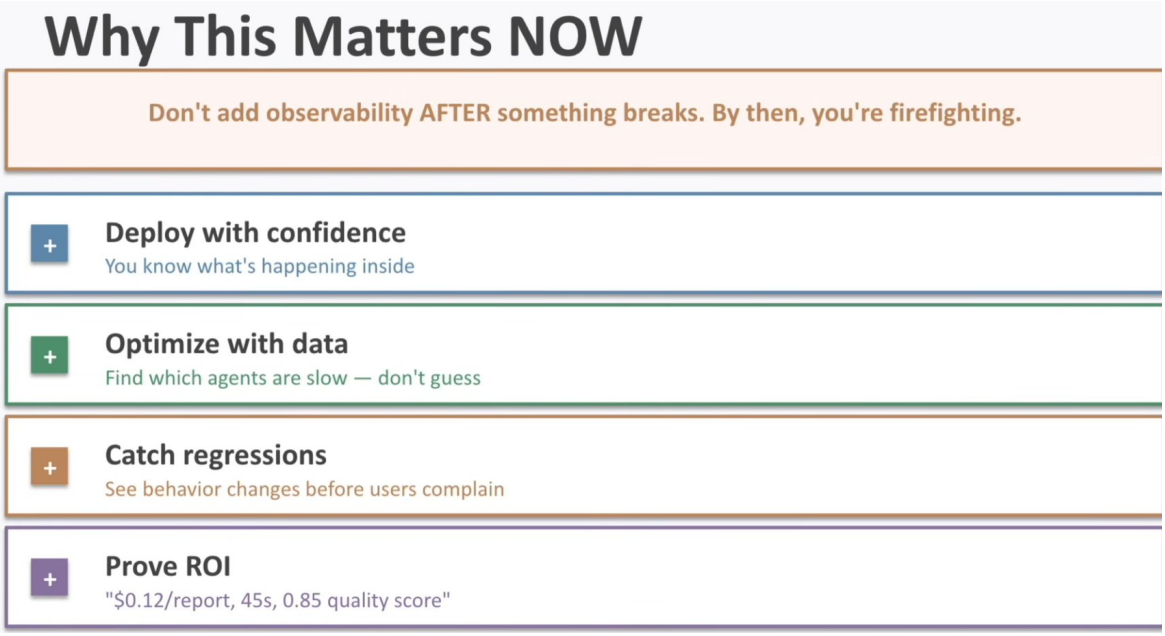


Figure 17: Observability pays off before problems, not after.

Beyond tracing: the production layers

In production, every LLM call passes through a few layers:

- **Security: input sanitization** blocks prompt-injection attempts such as "ignore your instructions". **PII protection** masks personal data (emails, phones, IBANs, AΦM) before it reaches the LLM or the logs. **Guards** check the answer before the user sees it.
- **Cost optimization: routing** sends easy questions to a cheap model and hard ones to a strong model. **Caching** and **token budgets** keep repeated and oversized requests cheap.
- **Error handling: retry** tries again after short API failures, waiting longer each time. A **circuit breaker** stops calling a broken service for a while instead of failing on every request. A **fallback** switches to a backup model when the main one is down.
- **Monitoring:** watches all the layers above without changing what they do.

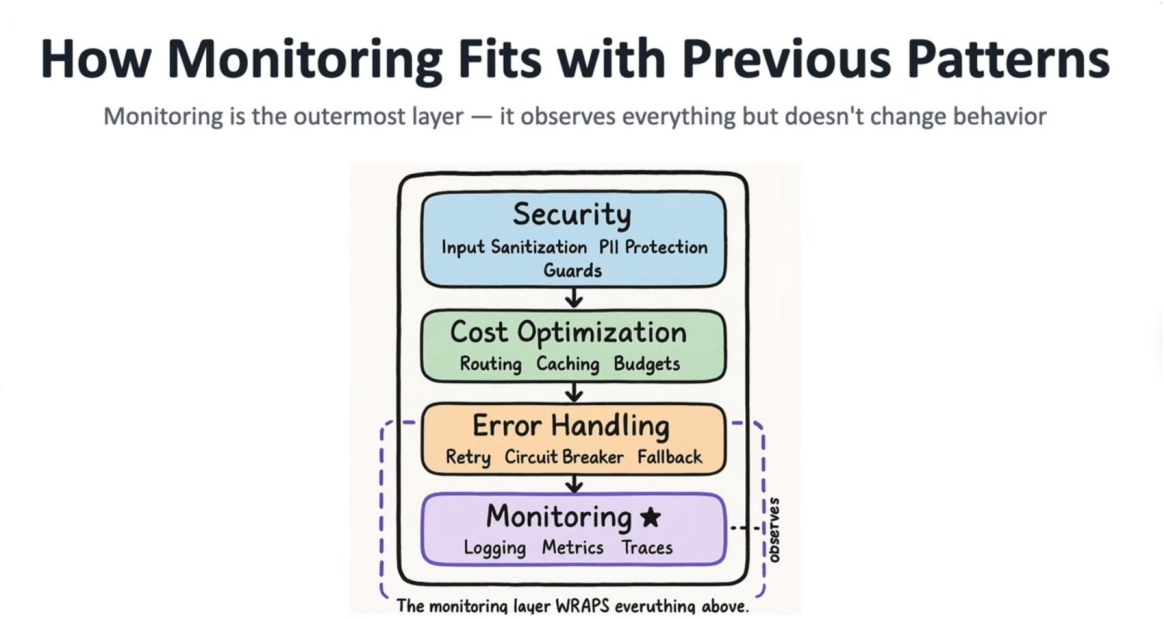


Figure 18: The production layers. Monitoring watches all of them.

Monitoring itself has three parts: **structured logs** (what happened, written as JSON so we can search them), **metrics** (how much: requests, latency, tokens, cost, as numbers for dashboards), and an **instrumented LLM** (a wrapper around every LLM call that records both).

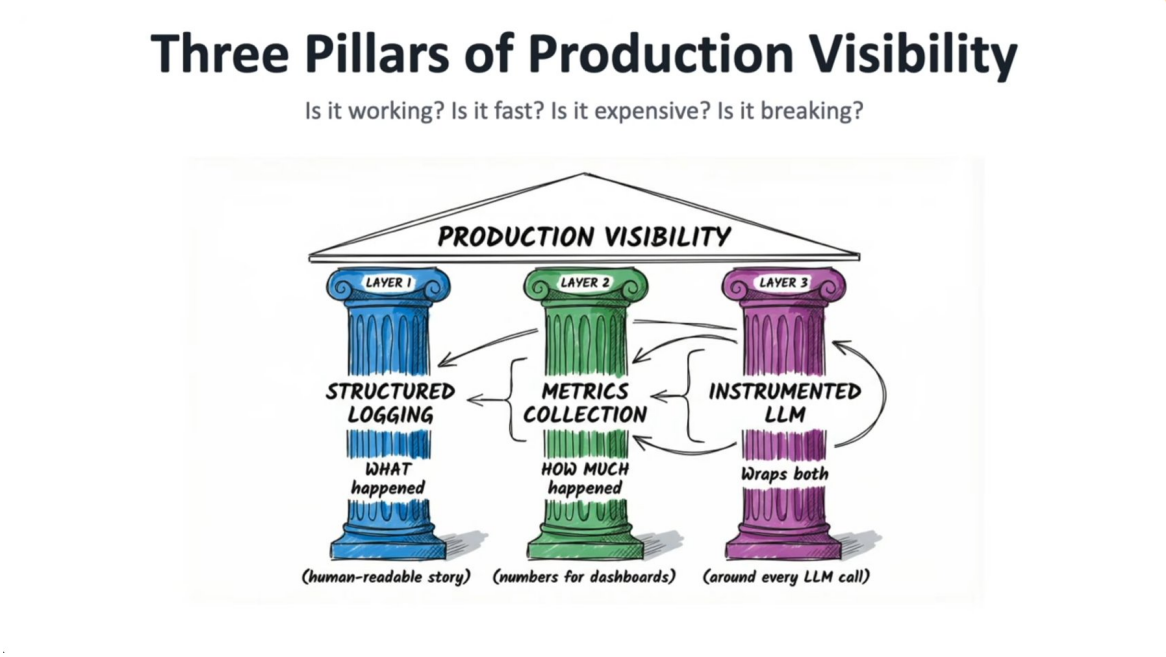


Figure 19: The three pillars of production visibility.

8. Conclusion

Most bad RAG answers come from five failures, and each has a clear fix:

Failure	Main fixes
Bad chunking	Right splitter, tuned size and overlap, parent-child chunks
Embedding mismatch	Hybrid search: vector + BM25, merged with RRF
Retrieval noise	Reranking, compression, multi-query, filters
Context overflow	Fewer chunks, compression, short history; token budgets for cost
Hallucination	Strict prompt, sources, refusal on weak retrieval, faithfulness checks

Start simple, measure retrieval on real questions, and add one fix at a time. Observability shows which failure we have, so we know which fix to use.